OUTPUT ANALYSIS

Hospital Patient Management System • Doubly Linked List, Stack and Array in C++

1. Time Complexity Analysis

Let **n** be the number of patient records currently stored in the doubly linked list.

Operation	Time Complexity	Reason
Admit Patient	**O(n)**	Duplicate-ID check scans the list; finding a free bed is a fixed 10-bed scan; insertion at *tail* is O(1).
Discharge Patient	**O(n)**	Searching the ID takes O(n); push to stack, freeing the bed and unlinking the node are O(1).
Search Patient	**O(n)**	Linear traversal from *head*; a match on ID or name may need every node (name search compares each one).
Display Forward / Backward	**O(n)**	Walks the list once using *next* from head or *prev* from tail, printing each node.
Bed Status	**O(1)**	Scans the fixed-size bed array (MAX_BEDS = 10), independent of the number of patients.
Undo Last Discharge	**O(n)**	Peek/pop on the stack is O(1); duplicate-ID check is O(n); re-insertion at tail is O(1).
Discharge History	**O(1)**	Reads at most MAX_STACK = 50 records from the top down to index 0, so it is bounded by a constant.

2. Space Complexity Analysis

Patient data is stored in dynamically allocated nodes. Each node holds the patient fields (ID, name, age, disease, bed number) and two pointers: *prev* and *next*.

Total Space Complexity	**O(n)**	The list holds n patient nodes (fields plus *prev* and *next*); the bed array and stack are fixed-size.
Bed Array	**O(1)**	Always MAX_BEDS = 10 elements, so it does not grow with the number of patients.
Discharge Stack	**O(1)**	Array-based stack of MAX_STACK = 50 records; the size is fixed at compile time.
Auxiliary Space per Operation	**O(1)**	Operations use a few temporary pointers and variables; admitting a patient allocates one extra node.

Interpretation: Operations that must locate a patient (admit, discharge, search, undo) are linear-time because the list is searched from *head*. Keeping a *tail* pointer makes insertion O(1) and a doubly linked list makes unlinking O(1) once the node is found. Bed allocation and discharge history are bounded by fixed array sizes, so they behave as constant time.